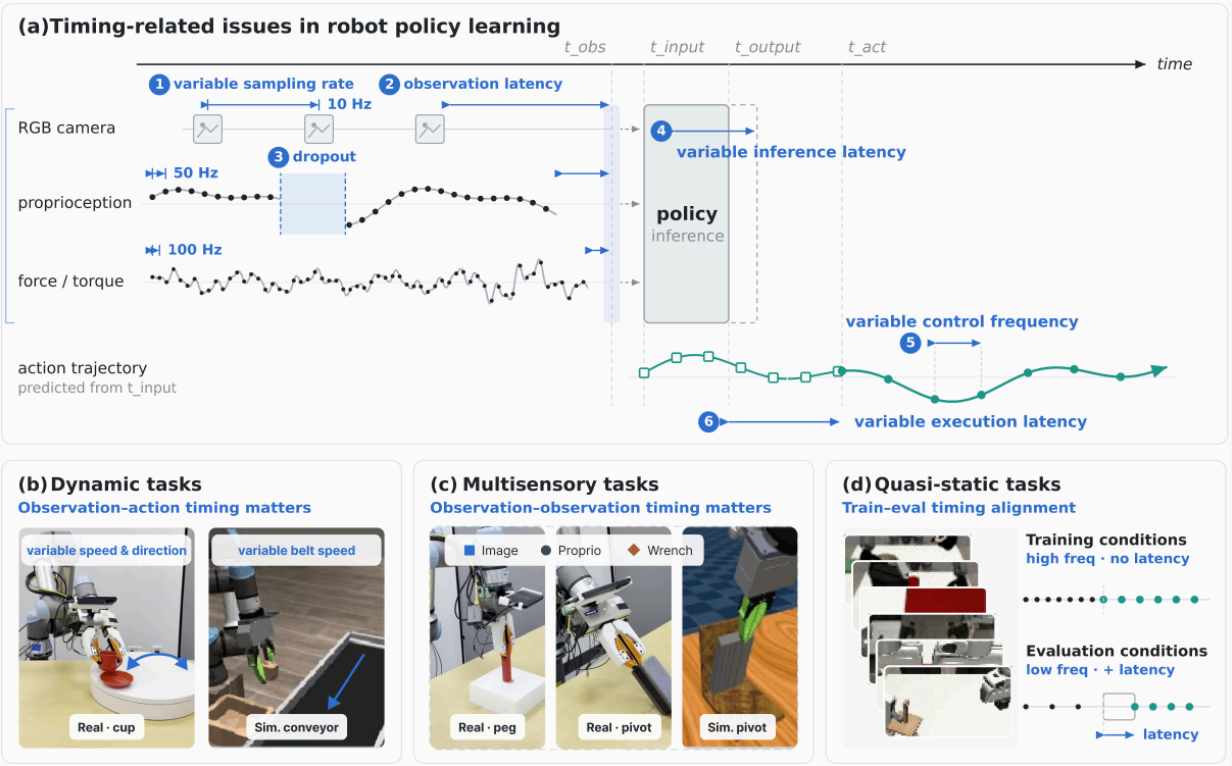
Chronos Policy: 분석 리포트

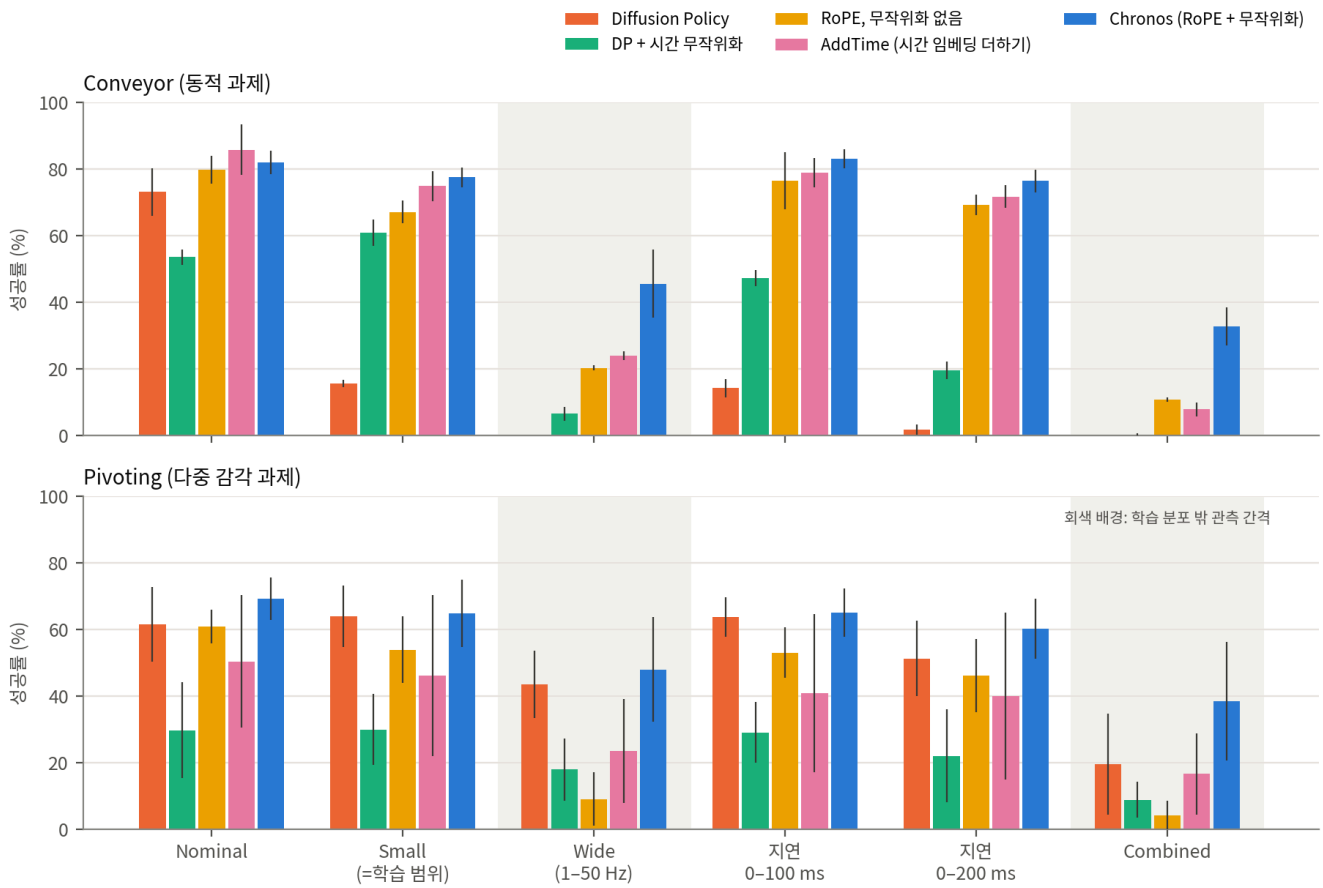
게시용 버전 · 원문 https://arxiv.org/abs/2610.13929 · arXiv:2610.13929v1 · Stanford University (Changhao Wang, Jaden Clark, Shuran Song) · 코드 미공개("Coming soon", 2026-10-28 확인) · 분석일 2026-10-28 · 논문 그림 © 저자, CC BY 4.0



논문 Fig. 1: 실제 로봇의 시간 변동 여섯 가지와, 과제 유형마다 중요한 시간 관계(동적: 관측-행동, 다중 감각: 관측-관측, 준정적: 학습-배포 차이).

핵심 요약

1. **무엇을 하는가:** 정책이 관측과 행동을 "시퀀스의 몇 번째"가 아니라 **관측 캡처 시각과 행동 실행 시각**으로 보게 했다. 위치 인코딩을 물리 시각으로 돌리는 time-aware RoPE와, 학습 때 관측 주기·지연·행동 시작 시각을 흔드는 시간 무작위화를 쓴다.
2. **숫자는 정확하다:** 논문 Table 3·4의 132칸이 프로젝트 페이지의 반올림 전 값과 모두 일치하고, 본문 수치 23개도 재계산과 맞는다. 시드별 성공 횟수도 132칸 모두 역산된다.
3. **동적 과제의 개선은 확실하다:** Conveyor에서 교란 5조건 모두 시드 단위 $p < 0.02$. 같은 무작위화로 학습한 DP와 비교해도 교란 평균 26.9 → 63.1이다.
4. **"Small" 조건은 Chronos의 학습 분포 그 자체다:** 주기·지연 범위가 학습 무작위화와 같다. DP 15.6 vs Chronos 77.6의 대부분은 학습 분포 차이이고, 시각 정보의 순수 효과는 DP + 무작위화 대비 +16.7%p(Conveyor), +34.9%p(Pivoting)다.
5. **약한 결론도 있다:** Pivoting의 Chronos-DP 차이와 RoPE-AddTime 차이는 시드 3개로는 유의하지 않다(p 0.13-0.92). 표준 6과제의 59.7 → 73.4는 세 과제에 몰려 있고, 나머지 세 과제는 81.2 vs 80.7이다. 표준 과제와 실제 로봇에는 DP + 무작위화 기준선이 없다.
6. **실제 로봇:** 12개 조건 모두 최고 점수지만 조건별 차이는 잡음 범위다. 과제별로 합치면 6개 비교 중 5개가 유의하다.



막대 = 3개 학습 시드 평균(시드당 150 에피소드), 선 = 시드 간 표준편차. 출처: 논문 Table 3, 프로젝트 페이지 data.js

우리 분석: Table 3을 시드 SD와 함께 다시 그림. Small 조건은 시간 무작위화 변형들에게 학습 분포 안이고, 회색 배경 조건은 모든 방법에게 분포 밖이다.

01. 논문 분석: Chronos Policy (arXiv:2610.13929)

- 제목: *Chronos Policy: Learning Robot Policies on Physical Time*
- 저자: Changhao Wang, Jaden Clark, Shuran Song (Stanford University)
- 공개: arXiv v1 2026-10-09 · 프로젝트 https://changhaowang.github.io/chronospolicy/ · 코드 "Coming soon"(2026-10-28 확인, 미공개)
- 라이선스: 논문 **CC BY 4.0** (arXiv abs 페이지에서 확인)
- 분석 범위: 본문 9쪽, 부록 A-C(총 18쪽), 프로젝트 페이지와 그 결과 데이터 파일(`data.js`, `dex-data.js`). 코드가 없으므로 검증은 표·페이지 수치 재계산과 numpy toy로 했습니다.

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 모방학습 정책이 관측과 행동을 "고정 주기 시퀀스의 몇 번째"로만 보던 것을, **관측을 찍은 시각과 행동을 실행할 시각(물리 시간)**으로 보게 바꿨습니다. 방법은 diffusion transformer의 위치 인코딩을 인덱스 대신 물리 시각으로 돌리는 **time-aware RoPE**와, 학습 때 관측 주기·지연·행동 시작 시각을 흔드는 **시간 무작위화**입니다.

정책은 다음과 같이 정의됩니다(Eq. 1, 3).

```
A_{t_a} ~ π_θ(· | O_{t_o}, t_o, t_a)
```

t_o는 각 관측의 캡처 시각, t_a는 요청한 행동 실행 시각입니다. 둘의 간격에 관측 주기, 측정 나이, 추론 지연, 실행 주기가 모두 담깁니다.

논문이 내세우는 기여는 세 가지입니다(§1).

1. 로봇 정책 학습을 물리 시간 위에서 정식화했습니다.
2. 비동기·다중 주기 관측을 고정 격자에 맞추지 않고 그대로 학습하는 time-aware RoPE diffusion transformer를 제안했습니다.
3. 현실적인 시간 변동을 넣은 벤치마크를 만들고, 동적 과제(관측-행동 시간)와 다중 감각 과제(관측-관측 시간)에서 개선을, 준정적 과제에서 성능 유지를 보였습니다.

---

2. Method

2.1 시간 변동의 종류 (Fig. 1)

논문은 실제 로봇의 시간 변동을 여섯 가지로 나눕니다: 센서 샘플링 주기, 관측 지연, 누락(dropout), 추론 지연, 제어 주기, 실행 지연. 과제 유형마다 중요한 시간 관계가 다릅니다.

과제 유형	중요한 시간 관계	예
동적	관측 시각 ↔ 행동이 효과를 내는 시각	움직이는 컨베이어, 회전하는 받침접시
다중 감각	서로 다른 관측의 캡처 시각	영상 20 Hz, 자세 20 Hz, 힘/토크 90-100 Hz
준정적	학습 때와 배포 때 시간 조건의 차이	DexMimicGen 6과제

2.2 물리 시간 정식화 (§3.1, Eq. 2-3)

요청 시각 t에서 H개 행동 chunk의 실행 시각을 다음처럼 둡니다.

$$t_{a,j} = t + d_t + j \cdot \Delta t_a,\ j = 0, \cdots, H-1$$

d_t는 첫 행동까지의 지연, Δt_a는 행동 간격(20 Hz → 50 ms)입니다. 다중 감각 입력은 모달리티마다 자기 캡처 시각을 그대로 유지합니다.

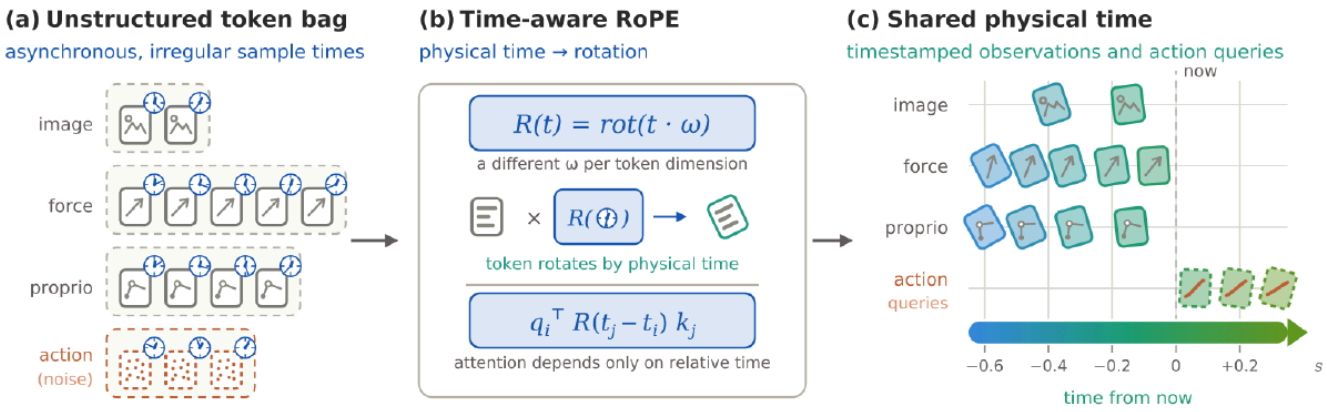
2.3 time-aware RoPE (§3.2, Eq. 4-6)

모든 토큰에 요청 시각 기준 상대 시각 $\tilde{t}_i = t_i - t$를 붙입니다(Eq. 4). 이미지 패치는 프레임 캡처 시각을, 자세·힘 토큰은 측정 시각을, 행동 토큰은 요청 실행 시각 $d_t + j\Delta t_a$를 받습니다. 이 값을 RoPE의 회전 좌표로 씁니다.

$$\tilde{q}_i = R(\tilde{t}_i)\, q_i,\ \tilde{k}_j = R(\tilde{t}_j)\, k_j \rightarrow \tilde{q}_i^{\top} \tilde{k}_j = q_i^{\top} R(t_j - t_i)\, k_j$$

따라서 attention은 **경과 시간 $t_j - t_i$에만** 의존하고, 시계 원점을 옮겨도 변하지 않습니다. 토큰 내용(embedding)은 건드리지 않고 시간은 attention의 위치 관계로만 들어갑니다. 저자들은 이 "내용과 시간의 분리"를 시간 임베딩을 토큰에 더하는 방식(AddTime)보다 강건한 이유로 듭니다.

구현 세부(부록 A.2): 주기 0.1-2 s 사이 로그 간격의 **16개 query-key 좌표쌍**을 회전합니다. 폭 512, head 8개라 head 차원은 64(32쌍)이므로, **head마다 절반만 시간으로 회전하고 나머지 절반은 시간과 무관한 채널**로 남는 것으로 보입니다(본문에 명시되지는 않았고, 숫자에서 추론한 것입니다. `03` §1).



논문 Fig. 2: 비동기·불규칙 토큰 → 물리 시각으로 회전하는 time-aware RoPE → 공유 물리 시간 축.

2.4 시간 무작위화 (§3.3, 부록 B.2)

학습 때 시연마다 다음을 무작위로 바꿉니다.

- **관측:** 스트림마다 주기 12.5-33.33 Hz와 전달 지연 0-50 ms를 따로 뽑습니다. 원본 데이터는 100 Hz입니다. Pivoting의 힘/토크는 1-100 Hz, 지연 0-100 ms입니다.
- **행동 시작 지연:** chunk의 절반은 지연 0, 나머지는 0-100 ms에서 뽑습니다.

뽑힌 측정은 원래 캡처 시각을 유지합니다. 그래서 같은 토큰 위치가 서로 다른 물리 간격에 대응해도 정책이 시각을 보고 해석할 수 있습니다. 부수 효과로, 고정 주기로 다운샘플링하면 버려질 측정까지 학습에 쓰게 되어 **데이터 증강**이 됩니다.

학습 목표는 표준 diffusion 손실입니다(Eq. 7-8). clean-action 예측과 $\ell 1$ 손실을 씁니다.

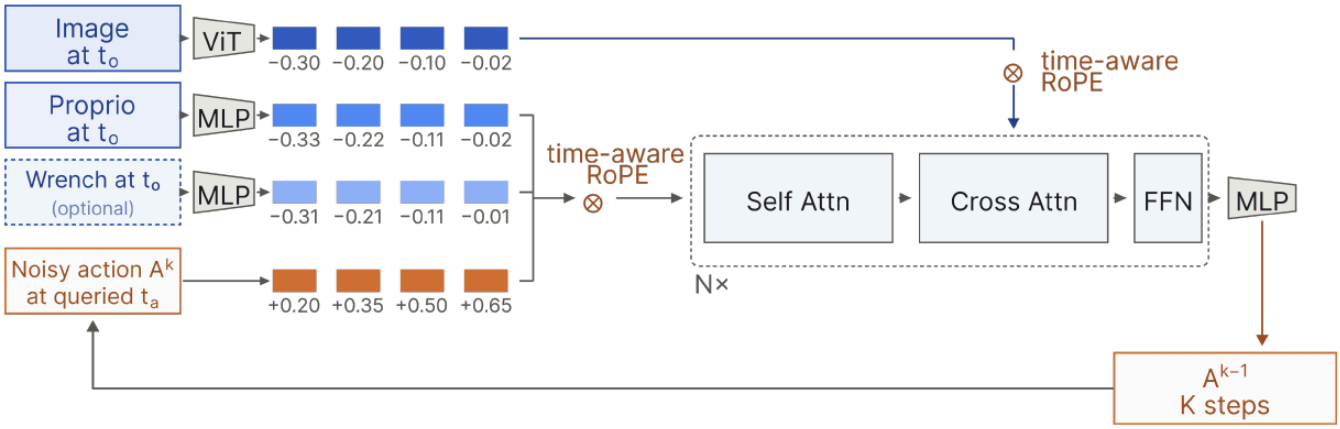
2.5 배포 (§3.4)

- chunk를 비동기로 실행합니다. 다음 chunk를 추론하는 동안 이전 chunk를 계속 실행합니다.
- 시뮬레이션에서는 주입한 추론 지연으로 d_t를 정합니다. 실제 로봇에서는 이전 요청들의 완료 시간으로 d_t를 추정합니다.
- 새 예측이 도착하면, 실행 시각이 이미 지난 행동은 버리고 나머지를 t_a에 맞춰 실행합니다.

2.6 구조와 학습 설정 (부록 A, Fig. 6)

항목	값
백본	16층 diffusion transformer, 폭 512, head 8 (DP와 Chronos 공통)
영상	SigLIP2 base, 256×256, 768차원 패치 특징 → 투영. 비영상 토큰끼리 self-attention, 영상 패치에는 cross-attention
관측 이력	Conveyor·표준 과제: 카메라당 영상 2장 + 자세 2개. Pivoting: 영상 2, 자세 3, 6차원 힘/토크 32개
행동	16개 @ 20 Hz. Pivoting은 19차원 adaptive-compliance 행동(Hou et al., 2025)
학습	AdamW 3e-4, 인코더 0.1배, BF16, 50 epoch, DDIM 24/8 step(시뮬레이션), 50/16 step(실제)
실행 길이	표준 12, Conveyor·Pivoting 8

3. Experiments



논문 Fig. 6: 자세·힘·잡음 행동 토큰의 self-attention, 이미지 패치 cross-attention, 양쪽에 time-aware RoPE.

3.1 설정 (§4.1, 부록 B)

- **시뮬레이션:** Conveyor(동적, 벨트 속도 무작위), Pivoting(다중 감각, 접촉), DexMimicGen 6과제(준정적: Threading, Assembly, Transport, Coffee, Drawer Cleanup, Pouring).
- **평가 조건 6개(Table 2, 3):**

조건	영상·자세 주기	힘/토크	관측 지연	추론 지연
Nominal	20 Hz	100 Hz	0	0
Small obs.	12.5-33.33 Hz	75-100 Hz	0-50 ms	0
Wide sensors (= Table 2의 "Large")	1-50 Hz	1-100 Hz	0-100 ms	0
Delay 0-100 / 0-200 ms	20 Hz	100 Hz	0	0-100 / 0-200 ms

조건	영상·자세 주기	힘/토크	관측 지연	추론 지연
Combined	Large	Large	0-100 ms	0-200 ms

- **통계:** 학습 시드 3개. Conveyor·Pivoting은 시드마다 평가 시드 3 × 50 에피소드(조건당 방법당 450 에피소드), 표준 과제는 시드마다 50 에피소드입니다. 체크포인트는 nominal 검증 성공률로 고릅니다(부록 B.1).
- **실제 로봇(부록 C):** Dynamic Cup(시연 252), Peg Insertion(220), Pivoting(322). 조건마다 10회(Pivoting 20회). 완전 성공 1점, 부분 성공 0.5점입니다.

3.2 동적 과제: Conveyor (§4.2, Table 3)

조건	DP	Chronos
Nominal	73.1	82.0
Small obs.	15.6	77.6
Wide sensors	0.0	45.6
Delay 0-100 ms	14.2	83.1
Delay 0-200 ms	1.8	76.4
Combined	0.0	32.7

시간 정보가 없는 DP는 관측 간격이나 추론 지연이 바뀌면 거의 실패합니다. 같은 영상 변위가 20 ms 간격이면 빠른 물체, 200 ms 간격이면 느린 물체이기 때문입니다.

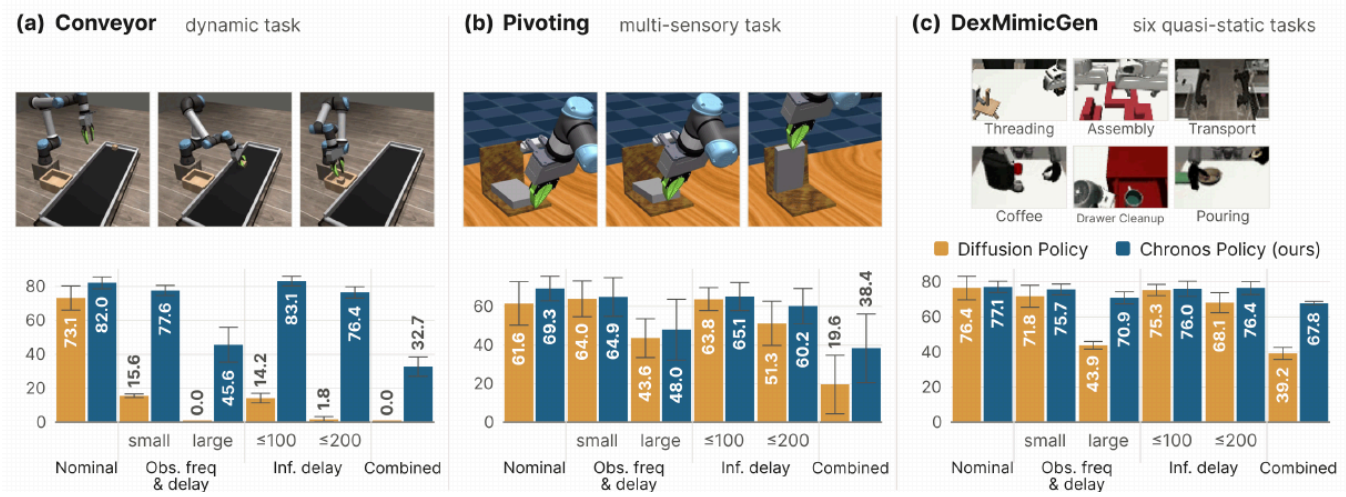
3.3 다중 감각 과제: Pivoting (§4.2)

작은 시간 변화에서는 비슷하고(64.0 vs 64.9), 큰 변화에서 차이가 벌어집니다: Delay 0-200 ms 51.3 → 60.2, Combined 19.6 → 38.4. 다만 시드 간 편차가 커서 이 차이는 시드 단위 검정에서 유의하지 않습니다(`03` §5).

3.4 무엇이 정책을 시간 인식형으로 만드는가 (§4.3, Fig. 3, Table 3)

변형	Conveyor 교란 평균	Pivoting 교란 평균
DP	6.3	48.4
DP + 시간 무작위화	26.9	21.6
RoPE, 무작위화 없음	48.8	33.3
AddTime (시간 임베딩 더하기)	51.5	33.5
Chronos (RoPE + 무작위화)	**63.1**	**55.3**

- 시간 무작위화만 하면 Conveyor에서는 도움이 되지만, Pivoting에서는 모든 조건에서 오히려 나빠집니다(nominal 61.6 → 29.8). 시각을 모르는 채로 주기를 섞으면 같은 토큰 위치가 서로 다른 모달리티 간 시간차를 뜻하게 되어 모호해진다는 해석입니다.
- 시각을 주면 무작위화가 일관되게 도움이 됩니다.
- AddTime은 Conveyor nominal에서만 RoPE보다 낫고(85.8 vs 82.0), 나머지 모든 조건에서 RoPE가 낫습니다.



논문 Fig. 4: 시뮬레이션 결과. (a) Conveyor (b) Pivoting (c) DexMimicGen 6과제 평균.

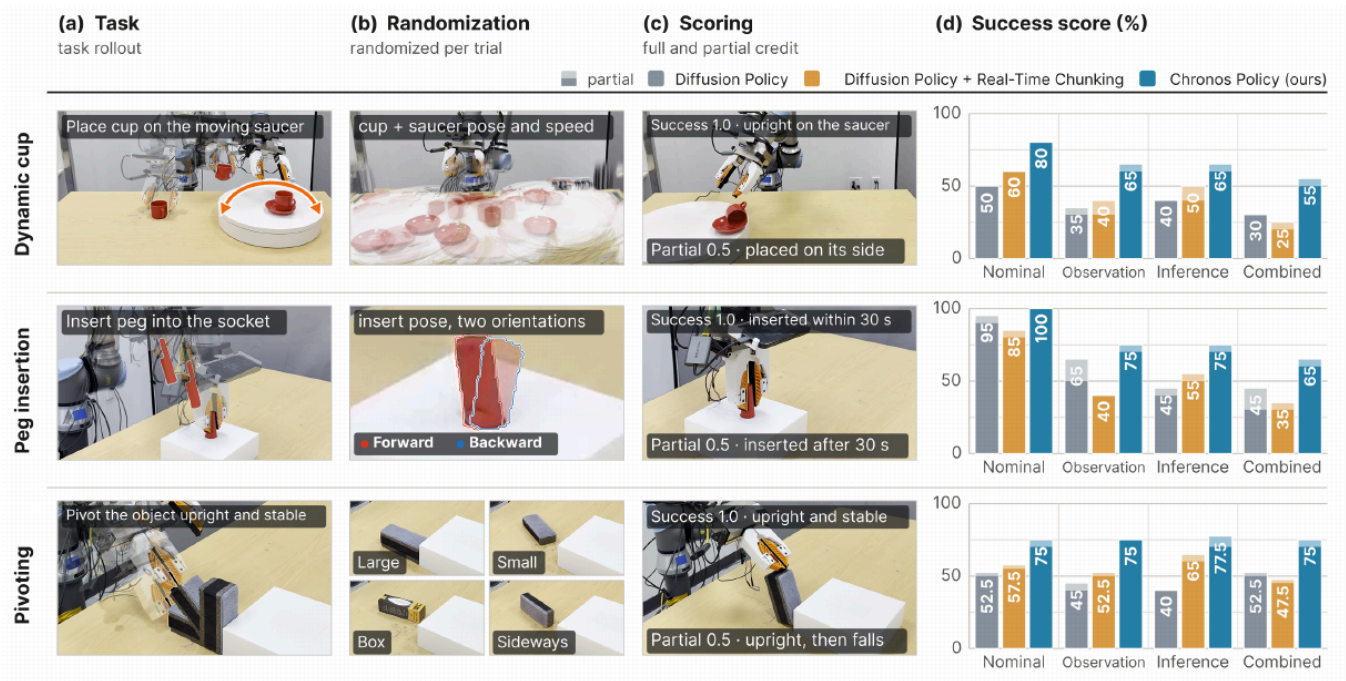
3.5 준정적 과제: DexMimicGen 6과제 (§4.4, Table 4)

nominal 평균은 76.4(DP) vs 77.1(Chronos)로 비슷하고, 교란 5조건 평균은 59.7 → 73.4입니다. 이득은 Assembly, Coffee, Pouring에 몰려 있습니다. 이 세 과제에서 DP가 Wide·Combined 조건에서 0-15%로 무너지기 때문입니다(`03` §5.3).

3.6 실제 로봇 (§4.5, Fig. 5, Table 7-9)

과제	정책	Nominal	Observation	Inference	Combined
Dynamic Cup (10회)	DP / RTC / **Chronos**	50 / 60 / **80**	35 / 40 / **65**	40 / 50 / **65**	30 / 25 / **55**
Peg Insertion (10회)	DP / RTC / **Chronos**	95 / 85 / **100**	65 / 40 / **75**	45 / 55 / **75**	45 / 35 / **65**
Pivoting (20회)	DP / RTC / **Chronos**	52.5 / 57.5 / **75**	45 / 52.5 / **75**	40 / 65 / **77.5**	52.5 / 47.5 / **75**

12개 조건 모두에서 Chronos의 점수가 가장 높습니다. RTC(Real-Time Chunking)는 추론 지연에서는 DP보다 낫지만, 관측 시간 변화는 다루지 못합니다.



논문 Fig. 5: 실제 로봇 세 과제와 조건별 점수. 밝은 막대는 부분 성공.

4. Limitations

4.1 저자가 밝힌 한계 (§5)

- 행동 주기는 고정(20 Hz)입니다. 가변 제어 주기와 시간 배율 조절(시연보다 빠르게나 느리게 실행)은 향후 과제로 남겼습니다.

4.2 분석하면서 추가로 발견한 한계와 의문점

1. **"Small obs." 조건은 Chronos의 학습 무작위화 범위와 정확히 같습니다**(12.5-33.33 Hz, 0-50 ms). Chronos에게는 학습 분포 안이고, 무작위화 없이 학습한 DP에게만 분포 밖입니다. 공정한 비교 상대는 DP + 무작위화입니다(Conveyor 60.9 vs 77.6, Pivoting 30.0 vs 64.9). 프로젝트 페이지 데이터도 이 조건을 `sensor_id`, Wide를 `sensor_ood`라고 부릅니다(`03` §3).
2. **표준 과제와 실제 로봇에는 "DP + 무작위화" 기준선이 없습니다.** Conveyor에서는 무작위화만으로도 교란 평균이 6.3 → 26.9로 올랐습니다. 그래서 표준 과제의 59.7 → 73.4, 실제 로봇의 차이 중 얼마가 시각 정보 덕이고 얼마가 데이터 증강 덕인지 나눌 수 없습니다.
3. **Pivoting과 RoPE-vs-AddTime 결론은 시드 3개로는 통계적으로 약합니다.** Pivoting의 Chronos-DP 차이는 6개 조건 모두에서 시드 단위 $p > 0.2$입니다. AddTime은 시드 하나만 잘 되고 둘은 무너진 패턴입니다(nominal 시드별 110, 61, 56/150). 이는 표현 방식의 차이라기보다 학습 불안정일 수 있습니다(`03` §5).
4. **실제 로봇은 조건당 10-20회입니다.** 12개 조건에서 모두 최고라는 서술은 맞지만, 조건별 차이 24개 중 permutation $p < 0.05$는 1개뿐입니다. 4개 조건을 합치면 과제별로 유의합니다(Peg vs DP만 $p = 0.09$).
5. **실제 로봇 학습의 무작위화 범위가 적혀 있지 않습니다.** 부록 B.2의 범위는 시뮬레이션 절에 있습니다. 실제 평가의 자세 주기 하한(8.33 Hz)과 힘/토크 범위(37.5-150 Hz)가 학습 범위 안인지 판단할 수 없습니다.
6. **이력 처리 방식이 불분명합니다.** 학습 때 힘/토크를 1-100 Hz로 뽑고 32개를 쓰면, 1 Hz에서는 이력이 31초에 이릅니다. 에피소드 시작 부분이나 오래된 측정을 어떻게 처리했는지 적혀 있지 않습니다.
7. **기준선 세부가 부족합니다.** DP가 어떤 인덱스 위치 인코딩을 쓰는지, 실제 로봇에서 d_t를 어떻게 추정하는지(이동 평균인지, 최댓값인지)가 없습니다.
8. **용어가 섞여 있습니다.** 같은 조건을 "Large"(Table 2), "Wide sensors"(Table 3), "large"(Fig. 3), `sensor_ood`(웹 데이터)로 부릅니다. Pivoting은 웹 데이터에서 `flipup`입니다. Table 3의 "Delay" 열은 관측 지연이 아니라 추론 지연입니다.
9. **코드가 공개되지 않았습니다.** 그래서 이 분석의 검증은 수치 정합성과 toy 수준입니다.

03. 재계산과 해석

GPU 없이 공개된 수치와 numpy toy로 검증했습니다.

- `analysis/table_check.py` → `table_check_results.json` (표·웹 데이터 대조, 본문 주장, 시드 역산, 검정)
- `analysis/timing_toy.py` → `timing_toy_results.json` (RoPE 성질, 시간 커널, 학습/평가 시간 범위)
- `analysis/plot_headline.py` → `report/figures/headline_ablation.png`, `standard_tasks.png`, `timing_coverage.png`

---

1. time-aware RoPE가 계산하는 것 (V1)

RoPE는 2차원 좌표쌍마다 각도 ω_k·t만큼 회전합니다. 회전행렬은 R(a)ᵀR(b) = R(b − a)를 만족하므로 다음이 성립합니다.

$$(R(t_i)\,q)^T\,(R(t_j)\,k) = q^T\,R(t_j - t_i)\,k$$

- **확인:** 무작위 q, k(32차원 = 16쌍)와 시각 1,000쌍으로 계산하면 두 식의 최대 오차는 7.8×10^{-14}입니다. 모든 시각을 ±100 s 옮겨도 내적은 최대 7.4×10^{-12}만큼만 변합니다. 즉 Eq. 6과 "시계 원점에 불변"이라는 서술은 정확합니다.
- **부분 회전으로 보입니다.** 부록 A.2는 "16개 query-key 좌표쌍을 회전한다"고 씁니다. 폭 512, head 8개면 head 차원은 64 = 32쌍입니다. 따라서 head마다 **절반은 시간으로 회전하고 절반은 회전하지 않는** 구성으로 보입니다(GPT-NeoX의 부분 rotary와 같은 형태). 회전하지 않는 절반은 시간과 무관하게 내용만으로 attention 점수를 만듭니다.
- **AddTime과의 차이에 대한 해석(가설):** AddTime은 시간 특징을 MLP에 통과시켜 토큰 자체에 더합니다. 그래서 학습 때 보지 못한 시간 차이에서는 토큰 내용이 예측할 수 없게 바뀝니다. RoPE는 내적의 시간 의존 부분만 바뀌고, 회전하지 않는 절반은 그대로입니다. 저자들의 "내용과 시간의 분리"가 구조적으로는 이 절반 채널로 구현된 것으로 보입니다. 다만 AddTime 결과 자체가 불안정해서(§5.3) 이 비교의 근거는 약합니다.

2. 시간 커널: 해상도와 되풀이 (V2)

16개 주기(0.1-2 s, 로그 간격)가 만드는 시간 유사도 s(Δt) = mean_k cos(ω_k Δt)를 계산했습니다(`timing_coverage.png` (b)).

항목	값	의미
s가 0.5로 떨어지는 Δt	**45 ms**	행동 간격(50 ms)과 비슷합니다. 인접 행동 토큰이 attention에서 구별됩니다
|Δt| ≥ 0.5 s에서 |s| 최대	0.34	멀리 떨어진 시각이 가까운 시각처럼 보이는 강한 되풀이는 없습니다
가장 느린 성분이 한 바퀴 도는 Δt	2 s	토큰 쌍 간격이 2 s를 넘으면 이 성분은 되풀이됩니다
간격이 2 s를 넘는 샘플 비율	Wide 0.5%, Combined 0.8%	되풀이는 성능 저하의 주원인이 아닙니다

처음에는 "Wide 조건에서 오래된 관측이 2 s 주기를 넘어 RoPE가 되풀이되기 때문에 성능이 떨어진다"는 가설을 세웠습니다. 계산해 보니 그런 샘플이 1% 미만이라 **이 가설은 기각합니다.** 주기 범위 선택은 이 실험의 시간 범위에 맞게 되어 있습니다.

3. 학습 때와 평가 때의 시간 범위 (V3)

부록 B.2(학습)와 Table 2(평가)의 설정을 시뮬레이션했습니다. 스트림마다 위상이 무작위인 균일 샘플링, 요청 시각까지 도착한 측정 중 최신 n개를 쓴다고 가정했습니다. 이 가정은 논문에 명시되지 않은 부분입니다.

조건	가장 오래된 이미지 나이 p99 / 최대	연속 이미지 간격 p99	학습 최대 간격(80 ms)을 넘는 비율	이미지 → 마지막 행동 간격 최대
학습	0.17 / 0.21 s	79 ms	0%	1.05 s
Nominal	0.10 / 0.10 s	50 ms	0%	0.85 s
Small obs.	0.17 / 0.21 s	79 ms	**0%**	0.96 s
Wide sensors	1.06 / 2.06 s	666 ms	**23.6%**	2.81 s
Delay 0-200 ms	0.10 / 0.10 s	50 ms	0%	1.05 s
Combined	1.06 / 2.06 s	675 ms	23.6%	2.93 s

여기서 세 가지가 나옵니다.

1. **Small obs.는 Chronos에게 학습 분포 그 자체입니다.** 주기 12.5-33.33 Hz와 지연 0-50 ms가 학습 무작위화 범위와 같습니다. 웹 데이터도 이 조건을 `sensor_id` 라고 부릅니다. 그러므로 "Small에서 DP 15.6 vs Chronos 77.6"(Conveyor)은 **같은 분포로 학습했는지의 차이**가 대부분입니다. 시각 정보의 기여는 같은 무작위화로 학습한 DP와 비교해야 합니다: Conveyor 60.9 → 77.6(+16.7), Pivoting 30.0 → 64.9(+34.9).
2. **Wide와 Combined는 모든 방법에게 분포 밖입니다.** 샘플의 23.6%는 학습 때 보지 못한 관측 간격(최대 약 1 s)을 갖고, 관측 나이는 학습 최대의 10배까지 늘어납니다. 이 조건에서 Chronos도 크게 떨어지는 것(Conveyor 77.6 → 45.6)은 ① 1 Hz 영상이 정보 자체를 잃는 것과 ② RoPE가 학습 범위 밖 시간차로 외삽하는 것이 겹친 결과로 보입니다. 둘을 나누려면 학습 무작위화를 1-50 Hz로 넓힌 실험이 필요합니다.
3. **Delay 0-200 ms도 절반은 분포 밖입니다.** 학습 때 행동 시작 지연은 최대 100 ms였으므로, U(0, 200 ms)에서 뽑힌 지연의 절반이 학습 범위를 넘습니다. 그래도 Conveyor에서 83.1 → 76.4로 비교적 덜 떨어집니다. 행동 토큰 시각의 외삽은 0.85 s → 0.95 s 정도로 작기 때문입니다.

힘/토크 이력: 학습 때 힘/토크를 1-100 Hz로 뽑고 32개를 쓰면, 이력 길이의 중앙값은 0.61 s이고 p99는 15.6 s입니다(1 Hz에서는 31 s). 에피소드 길이보다 길 수 있으므로 실제로는 잘라 내거나 채우는 처리가 있었을 텐데, 논문에 적혀 있지 않습니다.

4. 시간 정보가 없는 정책은 무엇을 틀리나 (V4)

Conveyor 직관을 수치로 옮겼습니다.

- **속도 추정 배율:** 학습 때 이미지 간격이 항상 50 ms였던 정책이, 실제 간격 g로 찍힌 두 영상을 보면 물체 속도를 (g / 50 ms)로 나눈 값만큼 틀리게 해석합니다. 그 배율의 p1-p99 범위는 Small에서 0.60-1.57배, Wide에서 0.40-13배입니다. 속도를 ±40-60% 잘못 보면 벨트 위 물체를 놓치기 쉽습니다. 이것이 DP의 Small 성능 붕괴(73.1 → 15.6)와 맞습니다.
- **추론 지연:** 지연 d_t를 모르면 목표 위치가 $v \cdot d_t$만큼 어긋납니다. 벨트 속도는 논문에 없으므로 0.1 m/s를 예로 들면, 0-200 ms 지연에서 평균 1 cm, 최대 2 cm입니다. 컵 받침이나 컨베이어 위 물체를 잡는 데는 충분히 큰 오차입니다.

5. 통계: 무엇이 확실하고 무엇이 약한가

5.1 시드별 성공 횟수 역산과 과분산 (S1)

웹 데이터에는 Table 3·4의 평균과 SD가 반올림 전 값으로 들어 있습니다. 시드당 에피소드 수(Conveyor·Pivoting 150, 표준 과제 50)로 나누어 떨어지는 성공 횟수 세 개를 찾으면 **132개 칸 모두 해가 있고**, 90개는 해가 유일합니다. 즉 표의 숫자는 보고한 실험 설계와 정확히 일치합니다.

역산한 시드별 값을 보면 시드 간 차이가 큽니다.

칸	시드별 성공 수	시드별 성공률
Pivoting, DP, nominal	78, 88, 111 / 150	52%, 59%, 74%
Pivoting, AddTime, nominal	56, 61, 110 / 150	37%, 41%, 73%
Pouring, DP, nominal	23, 46, 50 / 50	46%, 92%, 100%
Conveyor, Chronos, Wide	58, 61, 86 / 150	39%, 41%, 57%

관측 SD를 같은 성공률의 이항 SD(시드당 에피소드 수 기준)로 나눈 과분산 비율은 중앙값 1.29(IQR 0.89–2.33)이고, Pivoting에서는 2–3배입니다. 즉 **에피소드를 늘려도 줄지 않는 학습 시드 간 편차가 결과를 좌우**합니다. 그래서 검정의 단위는 에피소드(450회)가 아니라 학습 시드(3개)여야 합니다. 에피소드 단위 Fisher 검정은 p를 크게 과소평가합니다.

5.2 Conveyor: 확실함 (S2)

비교 (Chronos vs)	시드 단위 Welch p
DP	nominal 0.15, **교란 5조건 모두 < 0.02**
RoPE, 무작위화 없음	Small 0.016, Wide 0.050, Combined 0.020, 나머지 > 0.05
AddTime	Combined 0.010, Wide 0.066, 나머지 0.16–0.49

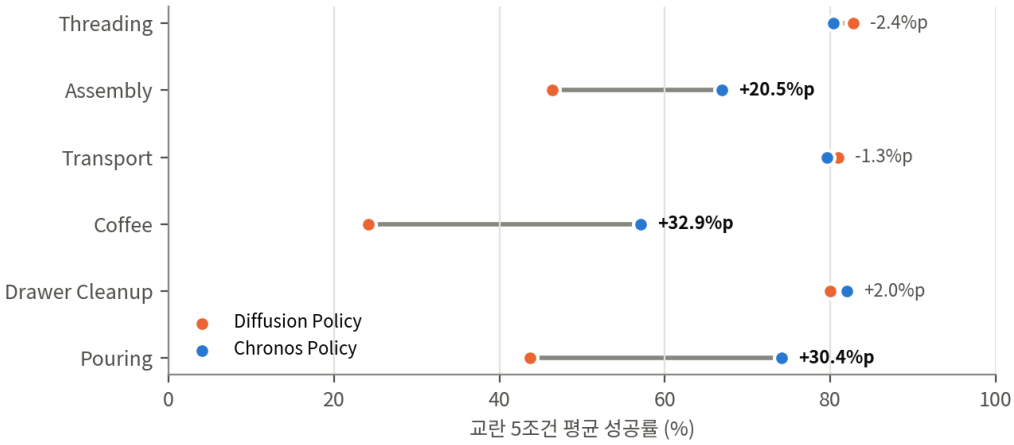
동적 과제에서 시간 정보가 강건성을 크게 높인다는 결론은 시드 3개로도 분명합니다. RoPE가 AddTime보다 낫다는 결론은 Combined에서만 유의합니다.

5.3 Pivoting: 방향은 같지만 유의하지 않음 (S3)

- Chronos vs DP: 6개 조건 모두 시드 단위 p = 0.24–0.92입니다. Combined 38.4 vs 19.6도 p = 0.24입니다.
- Chronos vs AddTime: 6개 조건 모두 p = 0.13–0.31입니다. AddTime은 nominal에서 시드 하나만 73%이고 둘은 37–41%입니다. 이 패턴은 표현 방식의 차이보다 **학습 불안정**에 가깝습니다.
- Chronos vs RoPE 무작위화 없음: Wide에서만 p = 0.03입니다.
- 반면 DP + 무작위화가 Pivoting을 해친다는 결과(nominal 61.6 → 29.8)는 효과가 커서 방향이 분명합니다.

따라서 "다중 감각 과제에서 시간 정보가 강건성을 높인다"(§4.2)와 "RoPE가 AddTime보다 낫다"(Pivoting 부분)는 **시드를 늘려 확인해야 할 주장**입니다.

5.4 표준 6과제: 이득이 세 과제에 몰림 (S4)



묶음	DP 교란 평균	Chronos 교란 평균
6과제 전체 (§4.4)	59.7	73.4
Assembly, Coffee, Pouring	38.1	66.0
Threading, Transport, Drawer Cleanup	**81.2**	**80.7**

- 이득은 DP가 Wide·Combined에서 무너지는 세 과제(Assembly Wide 13.3, Coffee Combined 2.7, Pouring Wide 2.0)에서 나옵니다. 이 칸들은 시드 단위로도 유의합니다(p 0.0001-0.014).
- 나머지 세 과제에서는 차이가 없고, 36칸 중 14칸에서는 Chronos가 더 낮습니다. Threading Combined는 77.3 → 63.3입니다(p = 0.08).
- 표준 과제에는 DP + 무작위화 기준선이 없습니다. Conveyor에서 무작위화만으로 교란 평균이 6.3 → 26.9로 올랐다는 점을 생각하면, 이 세 과제의 이득 중 상당 부분이 데이터 증강 효과일 수 있습니다.
- 평가 잡음도 큽니다. Coffee에서 Chronos는 Combined(60.0)가 nominal(51.3)보다 높습니다. 시드 3개 × 50회로는 ±10%p 안팎의 차이를 구분하기 어렵습니다.

5.5 실제 로봇: 과제별로 합치면 유의함 (S5)

시행 점수(1, 0.5, 0)로 조건 내 permutation 검정을 했습니다.

과제	vs DP: 차이(점), p	vs DP+RTC: 차이(점), p
Dynamic Cup (40회씩)	+27.5, 0.018	+22.5, 0.041
Peg Insertion (40회씩)	+16.2, 0.088	+25.0, 0.013
Pivoting (80회씩)	+28.1, 0.0002	+20.0, 0.0075

- 조건별(10회 또는 20회) 비교 24개 중 $p < 0.05$는 Pivoting 추론 지연 vs DP($p = 0.022$) 하나뿐입니다. 각 칸의 "최고 점수"는 잡음 범위 안의 차이입니다.
- 4개 조건을 합치면 6개 비교 중 5개가 유의합니다. 실제 로봇에서도 Chronos가 낫다는 결론은 과제 단위로는 지지됩니다.
- 다만 실제 기준선(DP, RTC)도 시간 무작위화 없이 학습한 것으로 보여, §5.4와 같은 교란 요인이 남습니다.

6. 정리: 각 비교가 실제로 분리하는 것

비교	분리하는 요인	결과의 강도
DP vs Chronos (Small)	무작위화 + 시각 정보 + 분포 안/밖	강함. 다만 대부분 "학습 분포" 효과
DP + 무작위화 vs Chronos	**시각 정보만**	Conveyor·Pivoting에서 큼(+16.7, +34.9). 이 저장소에서 가장 깨끗한 비교
RoPE 무작위화 없음 vs Chronos	무작위화만 (시각 있음)	Conveyor에서 유의, Pivoting은 Wide에서만
AddTime vs RoPE	시간 표현 방식만	Conveyor Combined에서만 유의. AddTime 학습 불안정
표준 과제, 실제 로봇	무작위화와 시각 정보가 섞임	크지만 원인 분리 불가

추가 검증 (2026-11-11)

스크립트: `analysis/extra_time_conditioning.py` → `analysis/extra_time_conditioning_results.json` (PyTorch CPU, 4 스레드, 실행 약 8분). 논문 모델이 아니라 1차원 컨베이어 toy에서 세 가지 시간 조건화 방식을 직접 구현했습니다.

추가 검증 1: 시간 조건화 방식과 학습 무작위화 범위가 불규칙 관측에서 어떻게 달라지는가

목적. 시간 조건화를 인덱스 위치, 토큰에 시간 임베딩을 더하기(AddTime), attention 안의 시간 회전(RoPE)으로 나누고, 학습 때 관측 주기·지연을 무작위화했는지와 범위에 따라 평가 조건(Small, Wide, 행동 지연)에서 위치 예측 오차가 어떻게 달라지는지 봅니다. Wide 저하가 학습 범위 밖 외삽 때문인지 관측 정보 자체의 손실 때문인지도 나누어 봅니다.

설정(가정).

- 물체 위치 $x(t) = x_0 + v \cdot t$, v는 0.05-0.3 m/s의 임의 부호(벨트 속도는 논문에 없어 가정). 정책은 최신 관측 2개(위치, 시각)와 요청 시각을 보고, 요청 시각 이후 16개 행동 시점(20 Hz, 0.8 s)의 위치를 최신 관측 대비 변위로 예측합니다.
- 조건화: index(학습된 위치 임베딩), addtime(정현파 시간 임베딩을 토큰에 더함), rope(attention의 q, k를 상대 시각으로 회전). 2층, 폭 64, 헤드 4, AdamW, 1500 step, 배치 128, 시드 3개.
- 학습 무작위화: fixed(20 Hz, 지연 0), rand(12.5-33.3 Hz, 캡처 지연 0-50 ms, 행동 시작 지연은 절반이 0, 나머지 0-100 ms), wide(1-50 Hz, 캡처 지연 0-100 ms, rand와 같은 행동 지연 혼합).
- 평가: nominal(20 Hz, 지연 0), small(rand 범위), wide(1-50 Hz, 지연 0-100 ms), delay200(20 Hz, 행동 시작 지연 0-200 ms). 시험 4096개 표본, 16 행동 시점의 평균 절대 오차(cm), 시드 3개 평균.
- 장치: CPU. 패키지: torch 2.15.0, numpy 1.26.4, Python 3.12.7.

결과 (평균 절대 오차, cm; 괄호 안은 시드 표준편차).

조건화	학습	nominal	small	wide	delay200
index	fixed	0.53 (0.26)	1.72 (0.06)	4.07 (0.14)	1.81 (0.09)
index	rand	1.46 (0.18)	1.42 (0.11)	2.68 (0.11)	1.45 (0.02)
index	wide	2.50 (0.17)	2.09 (0.12)	2.18 (0.02)	2.39 (0.05)
AddTime	fixed	0.29 (0.07)	3.58 (0.15)	7.54 (0.64)	4.88 (1.63)
AddTime	rand	1.88 (1.54)	2.03 (1.87)	4.37 (1.32)	2.60 (1.98)

조건화	학습	nominal	small	wide	delay200
AddTime	wide	1.65 (0.42)	1.22 (0.27)	1.55 (0.26)	2.50 (1.11)
RoPE	fixed	0.35 (0.05)	3.95 (2.15)	7.11 (2.70)	5.06 (0.89)
RoPE	rand	0.54 (0.07)	0.52 (0.02)	2.60 (0.06)	0.89 (0.06)
RoPE	wide	0.72 (0.22)	0.79 (0.21)	1.10 (0.26)	0.82 (0.26)

결론.

- 무작위화 없이 학습하면 세 방식 모두 학습 범위 밖 조건에서 크게 무너집니다. RoPE는 Small에서 0.35 → 3.95 cm, Wide에서 7.11 cm로 떨어집니다. 이 저하는 "학습 때 본 적 없는 주기·지연"의 결과입니다.
- 무작위화(rand)를 쓰면 RoPE가 세 방식 가운데 가장 안정적입니다(Small 0.52, delay200 0.89 cm). AddTime은 같은 설정에서 시드 간 편차가 커서(±1.9 cm) 방식의 차이라고 단정하기 어렵습니다. 3시드라 방식 순위는 약한 근거입니다.
- Wide 저하의 분해: RoPE에서 rand 학습 때 Wide 오차는 2.60 cm이고, 같은 방식을 1-50 Hz로 넓혀 학습하면 1.10 cm입니다. 학습 범위를 넓히면 대부분의 격차가 줄어드므로, rand 조건의 Wide 저하는 **주로 외삽**이라고 봅니다. 남은 1.10 cm와 nominal의 0.72 cm 차이는 1 Hz 영상의 정보 손실과 넓힌 학습의 비용이 섞인 값입니다. 둘을 엄밀히 나누려면 정보 손실이 없는 관측 조건(관측 시각을 알고 있는 oracle)이 추가로 필요합니다.
- 인덱스 위치는 무작위화가 nominal을 나쁘게 만듭니다(0.53 → 1.46 cm). RoPE는 같은 무작위화에서 0.35 → 0.54 cm로 비용이 작습니다. "시간을 명시적으로 알려 주면 무작위화 비용이 줄어든다"는 방향과 맞습니다.

한계. 1차원, 관측 2개, 이미지·힘 토큰 없음, 물체 속도와 행동 지연 분포는 가정입니다. 3시드이고 AddTime과 RoPE의 fixed 조건은 편차가 큽니다. 논문 Table 2의 실제 과제 수치와 직접 비교할 수 없습니다. 이 toy가 보인 것은 메커니즘(무작위화 범위, 외삽 대 정보 손실의 방향)이지 논문 수치의 재현이 아닙니다.

04. 주장 검증: 수치 정합성과 근거의 강도

코드가 공개되지 않아("Coming soon", 2026-10-28 확인) 논문 ↔ 코드 대조는 할 수 없습니다. 대신 논문 표, 본문, 프로젝트 페이지 데이터(`paper/site/data.js`, `dex-data.js`)를 서로 대조하고 재계산했습니다. 재실행: `python3 analysis/table_check.py && python3 analysis/timing_toy.py`

1. 수치 정합성

#	확인한 것	결과
V1	Table 3(60칸)·Table 4(72칸)의 mean ± SD vs 웹 데이터(반올림 전 값)	**132칸 모두 일치**
V2	실제 로봇 Table 7-9: 점수 = 100(F + 0.5P)/N, 시나리오 합계 = Overall, 웹 데이터의 점수·완전·부분 성공률	**모두 일치**
V3	본문 수치 23개(§4.2-4.4, 부록 B.3, Fig. 4c 조건별 평균 12개 포함)	**모두 일치**
V4	시드별 성공 횟수 역산(시드당 150 또는 50 에피소드)	132칸 모두 해 있음, 90칸은 해가 유일 → 보고한 실험 설계와 정합
V5	time-aware RoPE의 상대 시간 의존(Eq. 6)과 원점 이동 불변	수치 오차 10^{-12} 이하로 성립

숫자 자체에는 문제가 없습니다. 아래는 그 숫자가 각 주장을 얼마나 뒷받침하는지에 대한 판정입니다.

2. 주장별 판정

판정: ✅ 지지됨 · ⚠️ 부분적으로 지지되거나 해석에 주의 · ❌ 근거 부족

#	주장 (위치)	판정	근거
C1	동적 과제에서 시간 정보가 성능을 크게 높인다 (§4.2, Conveyor)	✅	교란 5조건 모두 시드 단위 Welch $p < 0.02$. 같은 무작위화로 학습한 DP와 비교해도 교란 평균 26.9 → 63.1
C2	다중 감각 과제에서 강건성을 높인다 (§4.2, Pivoting)	⚠️	방향은 같지만 6개 조건 모두 시드 단위 $p = 0.24$-0.92. 시드 간 편차가 이항 잡음의 2-3배 (`03` §5.3)
C3	시간 무작위화는 시각 정보가 있어야 효과적이다 (§4.3)	✅	시각 없는 무작위화는 Pivoting을 모든 조건에서 해침(nominal 61.6 → 29.8). 시각이 있으면 Conveyor 48.8 → 63.1 (Small·Wide·Combined에서 $p < 0.05$)
C4	time-aware RoPE가 AddTime보다 낫다 (§4.3)	⚠️	Conveyor Combined에서만 유의($p = 0.01$). Pivoting은 모든 조건에서 $p > 0.13$이고, AddTime은 시드 하나만 성공하는 불안정 패턴
C5	준정적 과제의 nominal 성능을 해치지 않는다 (§4.4)	✅	평균 76.4 vs 77.1. Assembly는 75.3 → 68.7로 낮지만 유의하지 않음
C6	준정적 과제에서 교란 평균 59.7 → 73.4 (§4.4)	⚠️	숫자는 정확함. 이득은 Assembly, Coffee, Pouring에 몰려 있고 나머지 세 과제는 81.2 vs 80.7. DP + 무작위화 기준선이 없어 원인 분리 불가 (`03` §5.4)
C7	실제 로봇 12개 조건 모두에서 최고 점수 (§4.5)	⚠️	서술은 정확함. 조건별 비교 24개 중 $p < 0.05$는 1개. 과제별로 합치면 6개 비교 중 5개 유의 (`03` §5.5)
C8	RTC는 추론 지연에는 도움이 되지만 관측 시간 변화에는 일관되지 않다 (§4.5)	✅	추론 지연에서 RTC > DP가 3과제 모두. 관측 조건에서는 Peg에서 RTC가 DP보다 낮음(40 vs 65)
C9	시간 무작위화가 버려질 측정까지 써서 증강 효과를 낸다 (§3.3)	✅ (정성)	시각 없이도 Conveyor 교란 평균 6.3 → 26.9. 다만 nominal은 73.1 → 53.6으로 떨어짐(부록 B.3에 명시)

3. 실험 설계에서 확인한 점

#	내용	영향
D1	**Small obs. = Chronos의 학습 무작위화 범위.** DP는 이 분포로 학습하지 않음 (`03` §3)	"Small에서 15.6 vs 77.6"은 대부분 학습 분포 차이. 시각 정보의 순수 효과는 DP + 무작위화와 비교해야 함(+16.7, Pivoting +34.9)
D2	**Wide·Combined는 모든 방법에게 분포 밖.** 23.6%의 샘플이 학습 최대 관측 간격(80 ms)을 넘고, 관측 나이는 학습 최대 0.21 s의 10배까지	Chronos의 Wide 성능(45.6)은 정보 손실과 RoPE 외삽이 섞인 값
D3	표준 과제와 실제 로봇에 **DP + 무작위화 기준선 없음**	C6, C7의 원인 분리 불가
D4	실제 로봇 학습의 무작위화 범위 미기재	실제 평가 조건(자세 8.33 Hz 등)이 학습 분포 안인지 판단 불가
D5	힘/토크 32개를 1-100 Hz로 뽑을 때의 이력 처리 미기재 (1 Hz면 31 s)	Pivoting 재현 시 구현 선택에 따라 결과가 달라질 수 있음
D6	체크포인트를 nominal 검증 성공률로 고름 (부록 B.1)	모든 방법에 같으므로 비교는 공정. 다만 nominal 수치는 약간 낙관적일 수 있음
D7	head 차원 64 중 16쌍만 회전하는 부분 rotary로 보임 (`03` §1)	재구현 시 rotary 비율을 명시해야 함. 논문에 직접 쓰여 있지 않음
D8	같은 조건의 이름이 여러 개(Large / Wide sensors / sensor_ood), Pivoting = 웹의 `flipup`, Table 3의 "Delay"는 추론 지연	읽을 때 혼동 주의
D9	Fig. 6의 시각 예시(이미지 4장, 행동 간격 0.15 s)는 본문 설정(이미지 2장, 50 ms)과 다름	설명용 그림으로 보임. 영향 없음

4. 구현 가능성 판단

4.1 공식 코드로 재현하는 경우

- 코드가 공개되지 않았습니다. Conveyor와 Pivoting은 저자들이 만든 벤치마크라 코드 없이는 재현할 수 없습니다.
- DexMimicGen 6과제는 공개 벤치마크라 기준선(DP)은 재현할 수 있습니다. 16층 DiT + SigLIP2, 시드 3개 × 6과제 × 2방법이면 GPU 수십-수백 시간이 필요할 것으로 추정합니다(논문에 계산량 미기재).

4.2 핵심 아이디어를 다시 구현하는 경우 (코드 없이 가능)

- time-aware RoPE는 기존 transformer 정책에서 **위치 인덱스를 상대 시각(초)으로 바꾸고, 주기 0.1-2 s 로그 간격 16쌍만 회전**하면 됩니다. 수십 줄 수준의 변경입니다.
- 시간 무작위화는 데이터 로더에서 스트림별 주기·지연을 뽑아 측정을 고르는 일입니다. **원본 데이터에 스트림별 캡처 타임스탬프가 있어야** 합니다.
- 작은 1차원 toy(움직이는 표적, 불규칙 관측)는 CPU로 학습할 수 있습니다. 이 기기에는 torch가 없어 설치가 필요합니다.

02. 선행연구와 계보

인용은 논문 참고문헌 기준입니다. 각 논문을 따로 열어 검증하지는 않았습니다. §4는 제 판단입니다.

1. 계보도

```
[고정 주기 모방학습]  Diffusion Policy, action chunking
        │  문제: 시퀀스 위치 = 시간이라는 가정. 지연·주기가 바뀌면 깨짐
        ├─▶ [배포 때 지연 보정]
        │     UMI 지연 맞춤 (Chi et al., 2024)
        │     비동기 chunk 실행 + prefix inpainting: RTC (Black et al., 2026), 학습 시 조건화
(Black et al., 2025)
        │     Bidirectional Decoding (Liu et al., 2025), masked chunking (Wang et al.,
2026a), DiscreteRTC (2026b)
        │     실행 시점 상태 예측: VLASH (Tang et al., 2025)
        │     측정된 지연을 조건으로: Delay-Aware DP (Liao et al., 2025)  ← 가장 가까운 정책 쪽 선행연구
        ├─▶ [추론 자체를 빠르게]  Diffusion Forcing (Chen et al., 2024), Streaming DP (Høeg et
al., 2024)
        └─▶ [다중 주기 감각]
              Reactive DP: 느린 영상 + 빠른 촉각 분리 (Xue et al., 2025), ImplicitRDP (Chen et
al., 2026)
              ManipForce: 주기 인식 임베딩 (Lee et al., 2025), LAG-Fusion (He et al., 2026), FA-
RDP (Zhuo et al., 2026)

[연속 시간 표현]  mTAN (2021), ContiFormer (2023), RoPE (Su et al., 2024)
        └─▶ 영상 생성: VideoRoPE (2025), VFRTok (2026), BulletTime: world-time RoPE (Wang et
al., 2026c) ← 가장 가까운 표현 쪽 선행연구

★ Chronos Policy: 관측 캡처 시각 + 행동 실행 시각을 RoPE 좌표로, 학습 시 시간 무작위화
```

차별점(저자 주장): 기존 연구는 지연 하나(추론 지연)나 주기 하나(모달리티별 속도)를 따로 다뤘습니다. Chronos는 관측 주기, 측정 지연, 모달리티 간 시간차, 추론·실행 지연을 **하나의 인터페이스(타임스탬프)**로 다룹니다. 시간을 행동 속도나 지속 시간 제어에 쓰는 연구(Rudin et al., 2022; Jia & Chen, 2025; CTCO; TARC)와 달리, "언제 관측했고 언제 실행할지"를 표현합니다.

2. 구성 요소별 출처

구성 요소	출처
정책 백본	Diffusion Policy 계열 transformer, DDIM 샘플러
영상 인코더	SigLIP2 base
회전 위치 인코딩	RoFormer/RoPE (Su et al., 2024)
AddTime 비교군	Fourier features (Tancik et al., 2020) + MLP
Pivoting 행동 표현	Adaptive Compliance Policy (Hou et al., 2025)
실제 기준선	RTC (Black et al., 2026)를 DDIM prefix inpainting으로 이식
표준 과제	DexMimicGen 6과제(평가 프로토콜도 원 논문 기준)
실제 로봇 데이터	저자 그룹의 UMI 계열 힘 센서 구성으로 보임(Choi et al., 2026 UMI-FT; Clark et al., 2026)

3. 이 저장소의 다른 논문과의 관계

- **do_as_i_do:** Do as I Do의 SAM 3D guided blending은 마지막 스텝에서 EMA 필터와 같았고 (`do_as_i_do/analysis/03_derivations.md`), 그 05의 A1은 "flow action head에 적응형 guided blending을 넣되 RTC를 확인한 뒤 신규성을 판단"이었습니다. Chronos는 RTC를 기준선으로 직접 비교하고 "비동기 실행과 시간 인식은 상보적"이라고 결론냅니다. 따라서 chunk 연속성 아이디어를 평가할 때는 **RTC와 Chronos식 타임스탬프를 둘 다 비교 대상으로** 두어야 합니다.
- **dexscrew:** DexScrew의 BC는 촉각과 관측 이력을 20 Hz 제어 스텝 단위로 넣고, 분석 결과 촉각보다 이력의 효과가 더 뚜렷했습니다(`dexscrew/analysis/01_paper_analysis.md`). 촉각은 보통 제어 주기보다 빠르게 들어오므로, Chronos의 Pivoting 설정처럼 "제어 격자에 맞추지 않고 캡처 시각과 함께 넣는" 대안을 시험해 볼 수 있습니다. 다만 Chronos에서도 다중 감각 쪽 개선은 시드 단위로 유의하지 않았습니다(`03` §5).
- **gpt6_astra_tactile:** 그 분석의 D2 가설은 느린 LLM 계층(약 0.05 Hz)이 전략을 정하고 빠른 정책(20 Hz 이상)이 촉각에 반응하는 구조이고, ablation에 느린 계층 지연 주입(0/5/20 s)이 있습니다. 느린 계층의 출력은 큰 지연을 두고 도착하므로, 그 출력에 "언제 만든 계획인지" 타임스탬프를 붙여 빠른 정책에 주는 것이 Chronos식 해법입니다.
- **hydra0:** Hydra-0은 이미지 평면 점 궤적(action flow)을 조건으로 영상을 생성합니다. Chronos가 가장 가깝다고 꼽은 BulletTime도 영상 생성에서 연속 시각을 RoPE로 넣습니다. world model과 정책이 같은 "물리 시각 좌표"를 공유하면, 서로 다른 프레임레이트의 데이터를 섞을 때 유리합니다.
- **flashdexretarget:** FlashDexRetarget은 K = 10개 미래 레퍼런스 프레임을 조건으로 씁니다. 사람 시연 데이터셋(TACO, OakInk2, HOT3D)은 프레임레이트가 서로 다릅니다. 레퍼런스 프레임을 인덱스가 아니라 물리 시각으로 주면 데이터셋 간 주기 차이를 흡수할 수 있습니다.

4. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

연구	관련성
ACT의 temporal ensembling (Zhao et al., 2023)	겹치는 chunk를 지수 가중으로 섞는 가장 단순한 chunk 연속성 기법입니다. RTC와 함께 비교 대상이 될 만합니다
Neural ODE/CDE (Chen et al., 2018; Kidger et al., 2020), GRU-D (Che et al., 2018)	불규칙 시계열을 연속 시간 모델로 다루는 고전적 접근입니다. 관측 간격 Δt를 모델 동역학에 직접 넣습니다
연속 시간 상태공간 모델(S4/S5/Mamba 계열)	이산화 간격 Δt를 입력마다 바꿀 수 있어, 불규칙 샘플링을 구조적으로 처리합니다. RoPE와 다른 방식의 "시간 인식"입니다
Time2Vec (Kazemi et al., 2019)	학습 가능한 주기 특징으로 시간을 임베딩합니다. AddTime 비교군과 같은 계열입니다
π0 계열의 비동기 추론 실무	RTC의 출처(Physical Intelligence)입니다. 대형 VLA에서 추론 지연이 수백 ms에 이르므로, Chronos식 타임스탬프의 효용이 더 클 수 있습니다